

CryptoChain Analyzer Dashboard - Final Report

Author: Ruben Elices Rodriguez

Course: Cryptography, UAX 2025-26, Prof. Jorge Calvo

Repository: https://github.com/rubenelices/BLOCKCHAIN_DASHBOARD_RER

Date: 14 May 2026

1. Project Overview

CryptoChain Analyzer is an interactive Streamlit dashboard that connects to the live Bitcoin blockchain and turns the main cryptographic mechanisms from Topic 7 into verifiable experiments. The application covers Proof of Work, block headers, compact difficulty targets, retargeting periods, Merkle roots and double-spend risk. It also adds an Artificial Intelligence layer for anomaly detection and supervised difficulty forecasting, so the project is not only a visual explorer but also an analytical tool built on real on-chain data.

The dashboard is implemented in Python with Streamlit, Plotly, pandas, scikit-learn and scipy. Live data is obtained without API keys from the Blockstream Esplora API for block headers, transactions and tip information, with Blockchain.info used as a secondary source for aggregated difficulty history. Network calls are centralized in `api/blockchain_client.py`, cached with `st.cache_data`, and protected with explicit error handling so temporary API failures are shown in the interface instead of crashing the app. The application auto-refreshes every 60 seconds and exposes seven modules from the sidebar: M1 Proof of Work Monitor, M2 Block Header Analyzer, M3 Difficulty History, **M4 AI Anomaly Detector**, M5 Merkle Proof Verifier, M6 Security Score and **M7 Difficulty Predictor**.

2. Cryptographic Verification

M1 - Proof of Work Monitor. The monitor retrieves recent Bitcoin blocks and decodes the compact `bits` field into the full 256-bit target using the standard coefficient/exponent representation. From that target it derives the number of leading zero bits required by the current difficulty and estimates network hash rate as $\text{difficulty} * 2^{32} / 600$, reported in EH/s. It also compares observed inter-arrival times with the theoretical exponential distribution implied by Bitcoin's Poisson mining model.

M2 - Block Header Analyzer. This is the strongest cryptographic validation module. For a selected block it downloads the raw 80-byte header, parses the six fields with the correct little-endian conventions, recomputes the double SHA-256 block hash and compares it with the block id returned by the API. It then reconstructs the target from `bits` and checks `hash_int < target`. The module therefore proves two essential facts directly in code: the displayed block id is the hash of the header, and the header satisfies the Proof of Work threshold. The dashboard also counts leading zero bits rather than only hexadecimal zeroes, which is the correct bit-level interpretation of the target.

M3 - Difficulty History. The difficulty module walks backwards through retargeting boundaries of 2016 blocks and reconstructs the adjustment sequence. For each period it plots difficulty against `actual_time / target_time`, where `target_time = 2016 * 600 s`. Ratios below 1 indicate that blocks arrived too quickly and difficulty should rise at the next retarget; ratios above 1 indicate the opposite. This connects the live data with Bitcoin's feedback control rule for keeping average block time close to 10 minutes.

M5 - Merkle Proof Verifier. Given a block, the verifier downloads the ordered transaction ids, rebuilds the Merkle tree locally with double SHA-256, reverses leaf byte order as Bitcoin requires, duplicates the last node on odd levels and compares the final root with the `merkle_root` parsed from the header. This demonstrates that a block header is not just a Proof of Work object: it also commits cryptographically to the complete transaction set, which is the basis of SPV-style verification.

M6 - Security Score. The security module translates mining difficulty into attack economics. It estimates the energy-only cost per hour of a 51% attack from network hashrate, miner efficiency and electricity price, and plots Nakamoto's double-spend success probability as a function of confirmations and attacker hash share. The result makes the usual "wait for confirmations" rule quantitative: deeper confirmations rapidly reduce success probability unless the attacker controls a very large fraction of hash power.

3. Artificial Intelligence Component

The mandatory AI component is implemented in **M4** as an anomaly detector over block inter-arrival times. Bitcoin mining is naturally modelled as a Poisson process with expected interval 600 seconds, so the baseline distribution for waiting times is exponential. Departures from this baseline may come from normal variance, mining-pool concentration, network latency or adversarial strategies such as selfish mining; the detector is therefore designed as a screening tool, not as a source of definitive labels.

The module builds a dataset from roughly the latest 2000 blocks, keeps strictly positive inter-arrival intervals and discards timestamp artefacts. It then applies two complementary methods. The statistical detector estimates `lambda_hat = 1 / mean(times)`, reports a Kolmogorov-Smirnov goodness-of-fit test, displays a QQ-plot and flags intervals outside the fitted 2.5% and 97.5% quantiles. In parallel, an IsolationForest from scikit-learn uses `log1p(inter_arrival)`, `height mod 2016` and UTC hour as features, allowing the model to capture contextual patterns that a one-dimensional threshold cannot.

Because real blocks do not contain ground-truth anomaly labels, the dashboard adds a controlled evaluation stage. It preserves the real chain backbone but injects synthetic anomalies: very short intervals of 5-30 seconds and very long intervals of 40-120 minutes. With these labels it reports precision, recall and F1-score for both detectors. This provides an honest validation loop: the statistical baseline is transparent and strong for extreme upper-tail events, while IsolationForest can better balance fast and contextual anomalies.

M7 adds a supervised counterpart. It builds a chronological dataset around difficulty retargets, predicts the next percentage difficulty change, and compares Linear Regression with a Random Forest regressor using a 70/30 temporal split. Features include previous difficulty, recent period ratios, previous percentage change and rolling means. The two AI modules therefore cover both retrospective anomaly detection and forward-looking prediction on the same Poisson-based mining process.

4. Engineering and Visual Design

The implementation is structured around reusable data functions, cached API access and module-specific visualizations. Plotly charts are chosen according to the data type: time series for difficulty, histograms with theoretical density for inter-arrival times, QQ-plots for distributional fit, heat maps and curves for double-spend probabilities, and compact metric cards for live network state. Axes are labelled with units such as EH/s, seconds and percentages, and the dark crypto palette is consistent across modules.

The project also includes defensive engineering decisions that matter in a live blockchain dashboard: public API rate limits are respected through caching, network errors are surfaced cleanly, parameters such as electricity price or attacker hash share are configurable, and the app can still explain partial results when one external endpoint is temporarily unavailable.

5. Conclusion

The final dashboard satisfies the core objective of the project: it verifies Bitcoin's cryptographic primitives from live data instead of only describing them theoretically. The most important result is the end-to-end validation path: raw header parsing, manual double SHA-256 hashing, target reconstruction, Proof-of-Work comparison and Merkle-root recomputation. On top of that, the AI modules provide statistically grounded anomaly detection and supervised difficulty forecasting. The system is therefore a complete educational and analytical tool for observing how Bitcoin security emerges from hash functions, probabilistic mining and economic cost.

References

- Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*.
- Eyal, I. and Sirer, E. G. (2014). *Majority is not Enough: Bitcoin Mining is Vulnerable*. Financial Cryptography and Data Security.
- Blockstream Esplora HTTP API documentation: <https://github.com/Blockstream/esplora/blob/master/API.md>
- Bitcoin Core developer reference, block chain section: https://developer.bitcoin.org/reference/block_chain.html
- Pedregosa, F. et al. (2011). *scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.